Submitted By:

Ahmad Ali Asad – 2025-CYS-30

Assignment:

CEP-1

Submitted To:

Sir Shahmeer Nawaz

For the fulfillment of,

Object-Oriented Programming Lab

Department of Computer Engineering

University of Engineering and Technology, Lahore

Pac-Man Game Lab Report:-

1. Introduction

For this lab, we had to create a basic version of the classic Pac-Man game that runs right in the console (no fancy graphics, just text). The whole point was to practice using classes and objects in Java. I liked this assignment because it felt like turning a real game idea into working code, step by step.

The game lets you move Pac-Man around a maze, eat dots to get points, and avoid two ghosts. You start with 3 lives, and the goal is to eat every dot without getting caught. If a ghost touches you, you lose a life and everything resets. Run out of lives and it's game over.

2. What the Program Does

The main goal was to build a playable game that shows:

- A fixed maze with walls and dots
- Player movement using keyboard (W/A/S/D)
- Random ghost movement
- Score tracking
- Lives system
- Collision detection
- Win and lose conditions

Everything is split into separate classes so the code stays organized and easy to understand (that's the OOP part we've been learning).

3. How the Code Is Structured

Here's a quick look at the main classes I used:

a. **Map.java:**
This holds the whole 9-row by 12-column maze. It stores walls (#), dots (.), and empty spaces. It also has methods to check if a spot is a wall, eat a dot, print the map, and tell when all dots are gone.

b. **PacMan.java:**
Controls where Pac-Man is and lets him move. He only moves if there's no wall in the way, and if he lands on a dot he eats it and adds 10 points.

c. **Ghost.java:**
There are two ghosts. They pick a random direction (up, down, left, right) each turn and move only if it's not a wall. They start at fixed spots.

d. **Position.java:**
A simple helper class that just keeps track of any (x, y) spot on the map and can compare two positions to see if they're the same.

e. **Score.java and Lives.java:**
Super straightforward — one adds up points, the other counts down your 3 lives and checks if the game is over.

f. **Gameboard.java:**

This is the “boss” class with the main method. It sets everything up, runs the game loop, prints the map every turn, reads your keyboard input, moves Pac-Man and the ghosts, checks for collisions, and handles winning or losing.

All the classes work together nicely. For example, when you move, PacMan tells the Map to eat the dot and tells the Score to add points.

4. How the Game Loop Works

The game runs in a while loop inside Gameboard.java:

1. Print the current map (shows “P” for you and “G” for ghosts).

2. Show your current score and lives.

3. Ask for your move (W/A/S/D or Q to quit).

4. Move Pac-Man.

5. Move both ghosts randomly.

6. Check if any ghost is on the same spot as Pac-Man. If yes, lose a life, reset positions, and check if you’re out of lives.

7. If you ate every dot, you win!

It feels pretty smooth when you play it. The map clears and redraws after every move so you always see the latest positions.

5. Testing and Results

I compiled and ran the program a bunch of times to test it.

- Moving around worked great — you can’t walk through walls.

- Eating dots added exactly 10 points each time and the dots disappeared.

- Ghosts moved in random directions but never went through walls.

- When a ghost caught me, it printed “GHOST CAUGHT YOU!”, took away one life, and put everyone back to starting spots.

- After eating all dots, it showed the win message with my final score.

- If I lost all 3 lives, it correctly ended the game.

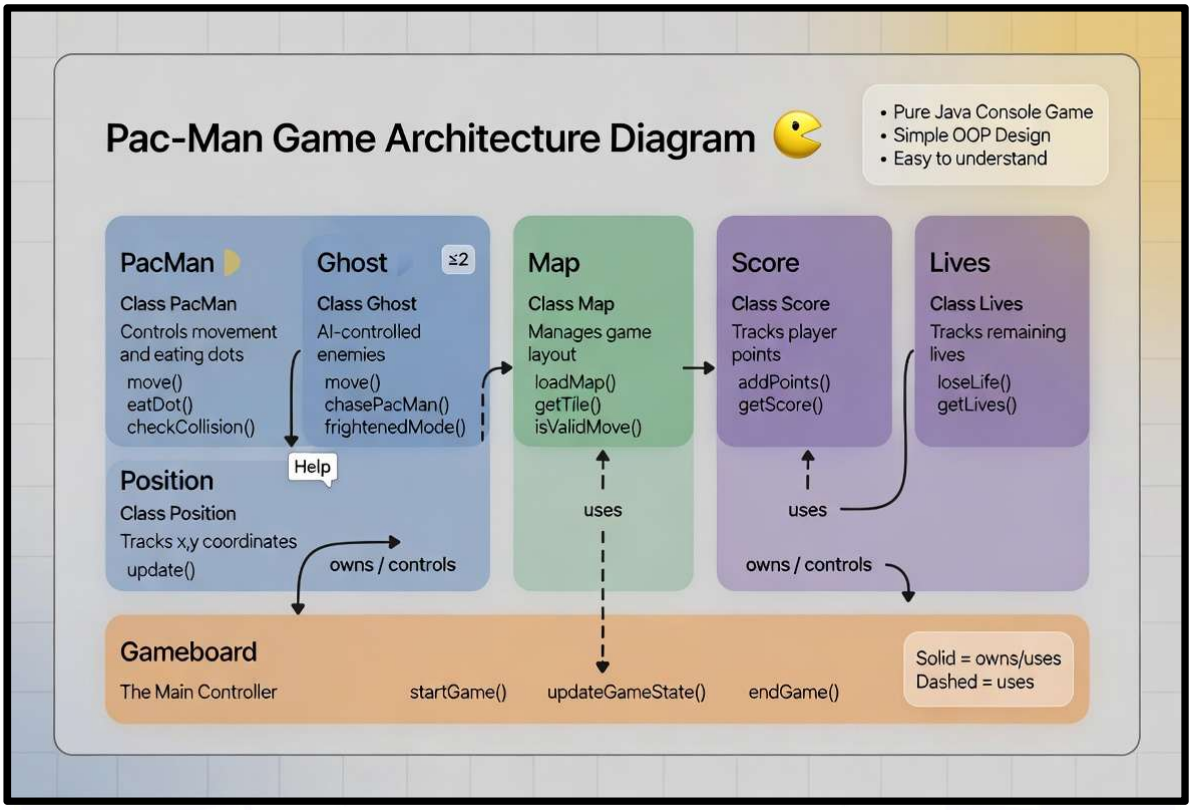
- Pressing Q let me quit early and still showed my score.

No crashes happened, and the score/lives updated perfectly every time. The game is actually pretty fun for something made only with text!

6. Challenges I Faced

The trickiest part was making sure the positions updated correctly and that the collision check happened after both Pac-Man and the ghosts moved. At first I kept forgetting to reset the ghosts when I lost a life, but once I added g1.resetPosition() and g2.resetPosition() it worked. Also, I had to be careful with the input — trimming spaces and making it uppercase so “w” and “W” both worked.

Conclusion

Overall, the Pac-Man game turned out better than I expected. It meets all the requirements, runs without bugs, and is actually enjoyable to play. I'm proud of how the different classes work together like a team. This project made OOP concepts click for me way more than just doing simple exercises.

Architecture Diagram:-